RA1 — Entorno de Desarrollo y Angular CLI

Angular CLI · Estructura · Terminal

1 · ANGULAR CLI (COMMAND LINE INTERFACE)

Comandos esenciales para gestionar proyectos de Angular.

```
# Instalar Angular CLI globalmente
npm install -g @angular/cli

# Crear un nuevo proyecto
ng new nombre-proyecto

# Iniciar servidor de desarrollo local (puerto 4200)
ng serve --open

# Generar elementos (Componentes, Servicios, etc)
ng generate component nombre-componente
ng g c componentes/header # Version corta
ng g s servicios/api      # Servicio
```

`ng serve` : Compila la app en memoria y recarga automáticamente el navegador al guardar cambios.

`ng generate (g)` : Crea archivos base (.ts, .html, .css, .spec.ts) y actualiza automáticamente el módulo correspondiente (app.module.ts).

2 · ESTRUCTURA DEL PROYECTO

Organización de archivos principales en Angular.

```
proyecto-angular/
├── src/
│   ├── app/
│   │   ├── app.component.ts   // Componente Raíz Lógica
│   │   ├── app.component.html // Plantilla Raíz
│   │   ├── app.module.ts      // Módulo Principal
│   │   └── app-routing.module.ts // Rutas Principales
│   ├── assets/                // Imágenes, iconos
│   ├── environments/          // Variables de entorno
│   ├── index.html             // HTML Principal
│   ├── main.ts                // Punto de entrada
│   └── styles.css             // Estilos Globales
├── angular.json               // Configuración del CLI
└── package.json               // Dependencias npm
```

RA2 — Componentes y Data Binding

Estructura @Component · Interpolación · Directivas (*ngIf, *ngFor) · @Input/@Output

1 · DECORADOR @COMPONENT

Estructura básica de un componente Angular.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-user',
  templateUrl: './user.component.html',
  styleUrls: ['./user.component.css']
})
export class UserComponent {
  public name: string = 'Achraf';

  cambiarNombre(): void {
    this.name = 'Nuevo Nombre';
  }
}
```

`@Component` : Marca la clase como componente e inyecta metadatos.

`selector` : Define el nombre de la etiqueta HTML personalizada (<app-user>).

`templateUrl` : Vincula el archivo HTML (vista).

`styleUrls` : Vincula los estilos específicos encapsulados.

2 · COMUNICACIÓN @INPUT / @OUTPUT

Paso de datos entre Padre e Hijo.

```
// Enviar datos del Padre al Hijo
@Input() titulo: string;

// Enviar evento del Hijo al Padre
@Output() notificar = new EventEmitter<number>();

enviarMensaje() {
  this.notificar.emit(42);
}
```

3 · DATA BINDING (VINCULACIÓN)

Comunicación entre el TypeScript y el HTML.

```
<!-- 1. Interpolación (TS -> HTML) -->
<h1>Hola, {{ name }}</h1>

<!-- 2. Property Binding (TS -> HTML) -->
<img [src]="imagenUrl" [disabled]="isDisabled">

<!-- 3. Event Binding (HTML -> TS) -->
<button (click)="cambiarNombre()">Actualizar</button>

<!-- 4. Two-Way Binding (TS <-> HTML) (Requiere FormsModule) -->
<input [(ngModel)]="name">
```

`{{ }} (Doble llave)` : Muestra valores de variables de TS en el HTML.

`[propiedad]` : Asigna una variable TS a un atributo HTML nativo.

`(evento)` : Ejecuta una función del TS al interactuar el usuario.

`[(ngModel)]` : Mantiene sincronizado el input HTML y la variable TS (caja en caja banana).

4 · DIRECTIVAS ESTRUCTURALES

```
<!-- Condicional if/else -->
<div *ngIf="isLoggedIn; else loginForm">
  Bienvenido usuario!
</div>
<ng-template #loginForm>
  Por favor inicia sesión.
</ng-template>

<!-- Bucle for -->
<ul>
  <li *ngFor="let user of usersList; let i = index">
    {{ i + 1 }} - {{ user.name }}
  </li>
</ul>
```

RA3 — Servicios, Datos y Formularios Reactivos

@Injectable · HttpClient · Observables · FormGroup

1 · SERVICIOS ANGULAR (@INJECTABLE)

Lógica centralizada y peticiones HTTP.

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DataService {

  constructor(private http: HttpClient) { }

  getUsers(): Observable<any> {
    return this.http.get('https://api.ejemplo.com/users');
  }
}
```

`@Injectable` : Indica que esta clase puede ser inyectada en constructores de componentes (Patrón Singleton).

`HttpClient` : Módulo de Angular para realizar llamadas API (GET, POST, PUT, DELETE). Devuelve Observables (RxJS).

2 · CONSUMIR UN SERVICIO

```
export class ListaComponent implements OnInit {
  public usuarios: any[] = [];

  // Inyectar el servicio en el constructor
  constructor(private dataService: DataService) {}

  ngOnInit() {
    // Suscribirse al Observable
    this.dataService.getUsers().subscribe({
      next: (data) => { this.usuarios = data; },
      error: (err) => { console.error(err); }
    });
  }
}
```

3 · FORMULARIOS REACTIVOS

ReactiveFormsModule.

```
import { FormGroup, FormControl, Validators } from '@angular/forms';

export class FormComponent {
  loginForm = new FormGroup({
    email: new FormControl('', [Validators.required, Validators.email]),
    password: new FormControl('', [Validators.minLength(6)])
  });

  onSubmit() {
    if (this.loginForm.valid) {
      console.log(this.loginForm.value);
    }
  }
}
```

`FormGroup & FormControl` : Permiten definir la estructura del formulario y validaciones completamente desde TypeScript.

`Enlace HTML` : El HTML se enlaza con `[formGroup]="loginForm"` y `formControlName="email"`.

RA4 — Navegación, Arquitectura y Guards

RouterLink · RouterModule · Lazy Loading · Guards

1 · ENRUTAMIENTO (APP-ROUTING.MODULE.TS)

Navegación entre componentes principales.

```
const routes: Routes = [
  { path: '', redirectTo: '/home', pathMatch: 'full' },
  { path: 'home', component: HomeComponent },
  { path: 'user/:id', component: UserDetailComponent },
  { path: '**', component: NotFoundComponent } // Error 404
];
```

`path` : La URL en el navegador.

`:id` : Parámetro dinámico en la URL (ej. /user/123).

`**` : Ruta comodín, se activa si ninguna otra coincide.

`<router-outlet>` : Etiqueta especial donde se renderizarán los componentes.

2 · NAVEGACIÓN HTML Y TS

```
<!-- Navegación HTML -->
<a routerLink="/home" routerLinkActive="clase-activa">Inicio</a>
<a [routerLink]="['/user', userId]">Perfil</a>

// Navegación desde TypeScript
constructor(private router: Router) {}

irAlPerfil(id: number) {
  this.router.navigate(['/user', id]);
}
```

3 · GUARDS Y LAZY LOADING

Protección de rutas y carga diferida (optimización).

```
const routes: Routes = [
  {
    path: 'admin',
    // Carga Diferida (Lazy Loading)
    loadChildren: () => import('./admin/admin.module')
      .then(m => m.AdminModule),
    // Guard para restringir acceso
    canActivate: [AuthGuard]
  }
];
```

`loadChildren` : No carga el módulo al iniciar la web, sino solo cuando el usuario entra a esa ruta, mejorando el rendimiento.

`canActivate (Guards)` : Funciones que devuelven true o false. Si devuelven false (ej. usuario no logueado), bloquean la entrada a la ruta.

★ Temario del Examen - OA01

QUÉ HAY QUE SABER ESCRIBIR / INTERPRETAR (Lista Oficial)

COMPONENTES Y VISTAS (SABER ESCRIBIR)

Decoradores, binding de datos y directivas.

```
// TypeScript (@Component)
@Input() title: string;
@Output() onSave = new EventEmitter<any>();

ngOnInit(): void { /* Al cargar */ }

// HTML Binding
<!-- Interpolación y Property Binding -->
<h1>{{ title }}</h1>
<img [src]="imgUrl" [disabled]="isSending">

<!-- Event Binding y Two-Way -->
<button (click)="onSave.emit($event)">Go</button>
<input [(ngModel)]="userName"> <!-- Ocupa FormsModule -->

<!-- Directivas Estructurales -->
<div *ngIf="isVisible; else noVisible">Visible</div>
<ng-template #noVisible>Oculto</ng-template>

<ul>
  <li *ngFor="let obj of array; index as i">
    {{ i }} - {{ obj.name }}
  </li>
</ul>

<!-- Clases dinámicas -->
<div [ngClass]="{'active': isActive, 'error': hasError}"></div>
```

■ BUG FRECUENTE: Usar `[(ngModel)]` sin haber importado `FormsModule` en `app.module.ts` dará error en la consola.

SERVICIOS, HTTP Y RXJS (SABER INTERPRETAR)

Inyección de dependencias, peticiones API y Observables.

```
// Servicio (@Injectable)
@Injectable({ providedIn: 'root' })
export class ApiService {
  constructor(private http: HttpClient) {}

  getUsers(): Observable<User[]> {
    return this.http.get<User[]>('api/users');
  }
}

// Consumo en el Componente (Suscripción a Observable)
constructor(private api: ApiService) {}

ngOnInit() {
  this.api.getUsers().subscribe({
    next: (data) => this.users = data,
    error: (err) => console.error(err)
  });
}
```

✓ ¡No olvides importar HttpClientModule en el app.module.ts para usar peticiones HTTP!

RUTAS Y GUARDS (SABER ESCRIBIR)

Configuración del enrutador y navegación dinámica.

```
// app-routing.module.ts
const routes: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'user/:id', component: UserComponent, canActivate: [AuthGuard] },
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: '**', component: NotFoundComponent } // 404 siempre al final!
];

// Router Outlet (HTML)
<router-outlet></router-outlet>

// Navegación HTML
<a routerLink="/home" routerLinkActive="active">Home</a>
<a [routerLink]="['/user', userId]">Perfil</a>

// Navegación TS (con ActivatedRoute)
constructor(private router: Router, private route: ActivatedRoute) {}

goToHome() { this.router.navigate(['/home']); }

ngOnInit() {
  // Leer parámetro :id
  this.route.params.subscribe(params => {
    const id = params['id'];
  });
}
```